

THETA GATE

An autonomous options agent that cannot place a trade it should not.

The model proposes a direction. Deterministic Python does everything else. A pure-function risk guard has the last word, and cannot be argued with.

The tension nobody else had to resolve

THE PROBLEM

Alpaca shipped their paper-trading skills on **25 August** — the day before this build started. Their own guidance requires a **human confirmation before every order**, and five operations demand it *regardless* of the unattended-mode setting:

```
cancel_all_orders  close_all_positions  close_position  
exercise_options_position  do_not_exercise_options_position
```

That is fundamentally incompatible with the autonomous agent this hackathon asks for. A cron at 10:30 on a Tuesday has nobody to ask.

“Every deployed path asserts paper at startup... There is no operator watching to catch a wrong endpoint, and a live account returns the same response shape as a paper one.” — Alpaca’s own skill

Confirmation is two things wearing one name

THE IDEA

A human confirmation does **legibility** (showing what is about to happen) and **authority** (deciding whether it may).

Alpaca automates away the second whenever a human is absent, and replaces it with an assertion that fails closed. **Theta Gate does the same thing, with eighteen assertions instead of one.**

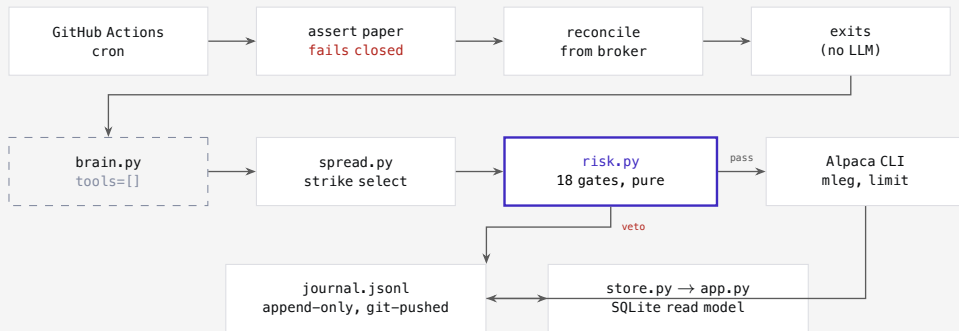
The one idea: the LLM has no tools at all. Every write goes through deterministic Python. The proposer suggests a direction; it never picks a strike, sizes a position, or holds an order credential.

The breach is *not reachable* — not merely forbidden by a prompt.

This is why the guard is pure functions with no I/O: it is the only component in the system holding a real decision.

Architecture

ONE TICK, EVERY FIVE MINUTES



The dashed box is the only non-deterministic component in the system, and it is the one with no network access, no credential, and no ability to write anything.

The LLM boundary, in full

BRAIN.PY

```
options = ClaudeAgentOptions(  
    system_prompt=SYSTEM_PROMPT, model=MODEL_ID, max_turns=1,  
    tools=[], allowed_tools=[], mcp_servers={},  
    strict_mcp_config=True, setting_sources=[],  
)
```

Zero tools. Zero MCP servers. One turn. No filesystem, user, or project settings. It sees only the same scalar market numbers already computed for the gates — no chain, no news, no broker credential.

It may return exactly five fields: `underlying`, `direction`, `confidence`, `thesis`, `invalidation`.

Why not a read-only allowlist? An allowlist still holds a network client and still depends on being maintained correctly forever. `tools=[]` cannot reach anything, and `strict_mcp_config=True` means a stray `.mcp.json` cannot quietly re-arm it.

Any failure — malformed JSON, schema violation, timeout, or a thesis that reads back an injected instruction — produces no proposal, never a crash.

The guard

RISK.PY --- PURE FUNCTIONS, NO I/O, TIME INJECTED

ENVIRONMENT

- paper asserted before *every* order
- kill-switch file; account status; options level ≥ 3

REGIME (entry-only, never blocks an exit)

- $VIX < 30$ and $VIX9D < VIX3M$
- $|\text{intraday move}| < 2\%$
- FOMC / CPI / PCE / NFP blackout

CONTRACT

- both legs need delta *and* IV — the ODTE guard
- $6 \leq DTE \leq 9$; short leg $0.16 \leq |\delta| \leq 0.25$
- quote age $\leq 60s$; spread $\leq 15\%$ of mid

PRICE

- credit within $\pm 40\%$ of $0.8 \times$ short delta
- absolute floor: credit $\geq 10\%$ of width
- ATM IV — 20d realised vol ≥ 2.0 points

SIZE AND EXPOSURE

- max loss $\leq \$1,000$; open risk $\leq \$3,000$
- ≤ 2 open, ≤ 1 per underlying
- buying power $\geq \$25,000$ *and* $\geq 5 \times$ max loss

DRAWDOWN

- -1% on the day $\rightarrow$ no new entries
- equity $\leq \$98,000 \rightarrow$ HALT and close out

First rejection wins and is final. No gate is re-evaluated after a veto, and no LLM sits downstream of one. Every number lives in [governance.json](#), rendered verbatim on the dashboard beside the line “no LLM can write to this file”.

The strategy, and its honest edge

WHAT IT ACTUALLY TRADES

Short credit vertical on SPY or QQQ. \$5 wide. 6–9 DTE. Short leg at 0.16–0.25 delta. Exactly one contract, all week.

WHY TWO LEGS, NEVER A CONDOR

Half the legs, half the fills to win. Roughly 10% of eligible paper fills come back partial, and a 4-leg order that half-fills *manufactures a naked short*. Non-negotiable.

WHY NOT 0DTE

0DTE has no greeks and no IV, so a delta-targeting rule has no input at all. This is structural, not a preference.

There is no arithmetic edge, and we say so.

Sweeping every strike from 0.15 to 0.45 delta across five widths, expected value is **negative in every case** — between $-\$4$ and $-\$8$ per contract, precisely the bid-ask cost. Delta *is* the risk-neutral probability, so a fairly priced chain cannot yield edge by arithmetic.

The only defensible claim is the **variance risk premium**: implied vol has historically exceeded realised. Real, thin, and **six sessions cannot measure it**.

The LLM is a direction filter and a veto. It is not the edge.

Three things live testing corrected

ONE REAL SPREAD, OPENED AND CLOSED, 26 AUGUST

1. The original credit gate would have vetoed every trade. We specified credit/width of 0.20–0.45. Observed on SPY at 765.55, 7 DTE, short leg at -0.197 delta: net credit **0.60** — a ratio of **0.120**. Widening makes it worse, monotonically. Across the chain the real relationship is **credit/width** $\approx 0.8 \times$ **short delta**. Recalibrated to a relative test.

2. Margin held is the full width, not the max loss. The fill consumed exactly **\$500** of options buying power on one \$5-wide contract — width $\times 100 \times$ qty — while max loss was \$443. Two different numbers; the buying-power gate used the wrong one.

3. A duplicate `client_order_id` is rejected, never duplicated. Alpaca returns HTTP 422. That single verified fact is the entire idempotency mechanism — it is what lets a crashed tick re-run safely with *no database at all*.

Every one of these was a plan that read fine and was wrong.

Durability without a database

WHAT WE DELIBERATELY DID NOT BUILD

The canonical design called for Postgres with row-level security, six credentialed roles, OIDC/Sigstore attestation, fenced distributed leases and hash-chained event sourcing. That is weeks of distributed-systems work. **We cut it, on the record, and said why.**

WHAT REPLACED IT

- **Idempotency from the broker.** A deterministic `client_order_id`, always looked up before submitting.
- **Concurrency** is one Actions `concurrency:` group — a platform feature.
- **The broker is the source of truth**, refetched every tick.
- **The journal** is append-only JSONL, written *before* the network call that might submit.

SQLite, but only downstream.

The dashboard needs aggregates, so `store.py` replays the journal into SQLite with a `sha256` hash chain, making an edited history detectable. It is a pure *function* of the journal — rebuilt from line 1, gitignored, never authoritative. A binary `.db` in the git-publish path would collide with the one durability mechanism we actually have. JSONL merges; a database file does not.

What the agent did

RESULTS --- PAPER ACCOUNT, FRESH, \$100,000

REALISED P&L
[P&L]

TRADES CLOSED
[n]

WIN RATE
[%]

MAX DRAWDOWN
[%]

equity curve --- generated Thursday 3 Sep from data/journal.jsonl

Every trade above was placed by the agent. The account has never received a manual order — not one, not even a test. Its history *is* the evidence of autonomy, so every live experiment in this deck used a separate throwaway account.

What six sessions can and cannot show

THE PART MOST DECKS LEAVE OUT

The sample is **[n]** trades across six sessions. That is statistically nothing, and we are not going to claim otherwise.

WHAT THIS CANNOT DEMONSTRATE

- That the strategy has edge. At -0.197 delta the breakeven win rate is 88% against a risk-neutral 80%.
- The sign of the P&L. One 2% adverse gap inside six sessions erases the book.

WHAT IT CAN DEMONSTRATE

- That the guard held: which gates fired, how often, and that no trade was placed outside them.
- That loss is capped by construction — max loss is the width, fixed at entry. A dropped cron run cannot exceed it.
- That the agent ran unattended, reconstructable from a committed, hash-chained journal.

The number worth judging is not the P&L. It is whether the risk guard held — and that is measurable at $n = \mathbf{[n]}$, whereas edge is not.

Built, and what it cost

ENGINEERING

THE SYSTEM

- `risk.py` — 18 gates, pure functions, no I/O, time injected
- `spread.py` — deterministic strike selection and mleg construction
- `loop.py` — one tick; recovery, reconcile, exits, entry, journal
- `brain.py` — the single bounded model call
- `store.py` / `app.py` — read model and live dashboard

THE INTEGRATION

Alpaca CLI for every broker call. No `alpaca-py`, by choice.

The honest failure mode: roughly 4:1 against — about eight \$100 wins per one \$800 loss. The gates cap the left tail; they do not change the sign of a week.

Nothing here is claimed untested.

Where a fact could be verified against the live API, it was — the 422 on a duplicate order id, the margin arithmetic, the credit/delta curve, the `alpaca doctor --profile` flag that silently lies.

An adversarial review pass over `loop.py` caught six real defects, two of them blockers, before any of it was trusted with an order.

THETA GATE

The risk guard is what replaces
the human confirmation step.

It is the thing you must build if you want autonomy without recklessness —
and it is the gap Alpaca's own tooling leaves open.

PAPER TRADING ONLY • NO MANUAL ORDERS, EVER • FULL JOURNAL COMMITTED TO GIT